# Nesne Yönelimli Programlama – Konu Anlatımı

Unified Process, IEEE SPMP ve Süreç İyileştirme

Slaytlar, ders kitabı (Schach 8e), ders videoları ve WhatsApp grubu temel alınmıştır

9 Kasım 2026

## 3 Unified Process (Chapter 3)

**! Hoca derste vurguladı Hafta 1 ve 4**

*"One of the most important chapters is third chapter because the unified process is described here ... These phases and workflows of unified process is very important in terms of our course."* Eski vize notlarında: *"Unified Process nedir? Fazlarını ve iş akışlarını açıklayınız — bunları sınavda soruyorum."* IEEE SPMP için: *"Bu formatı önemsiyorum ve genelde sınavlarda da soruyorum, başlıklarını soruyorum ... 1, 2, 3, 4, 5, 6, 7, 8. ana başlıkları."*

### 3.1 Unified Process nedir?

1990'ların en başarılı üç nesne yönelimli yöntemi Booch'un yöntemi, Jacobson'un Objectory'si ve Rumbaugh'un OMT'siydi. 1999'da üçü bu yöntemleri birleştirerek tam bir nesne yönelimli analiz ve tasarım metodolojisi yayımladı: önce *Rational Unified Process (RUP)*, sonra *Unified Software Development Process (USDP)*, bugün kısaca **Unified Process**.

- Unified Process **nesne yönelimli bir yazılım geliştirme metodolojisidir**.
- Bir yazılım üretmek için **bir dizi adım değildir**; "herkese uyan tek" bir metodoloji olamaz, çünkü çok farklı türde yazılımlar vardır.
- **Uyarlanabilir (adaptable)** bir metodolojidir: geliştirilecek ürüne göre değiştirilmelidir.
- Bir **modelleme tekniğidir**: model, ürünün çeşitli yönlerini gösteren UML diyagramları kümesidir. **UML (Unified Modeling Language)** hedef ürünü modellemek için kullanılan araçtır. UML grafiktir: "bir resim bin kelimeye bedeldir"; yazılımcıların hızlı ve doğru iletişim kurmasını sağlar.
- Nesne yönelimli paradigma doğası gereği **yinelemeli ve artırımlıdır**: UML diyagramları tatmin edici olana kadar tekrar tekrar yineleme ve artırım yapılır.

**Sınav cevabı kalıbı: "What is the Unified Process?"**

*The Unified Process is an object-oriented software development methodology (Booch, Jacobson, Rumbaugh, 1999). It is not a series of steps for constructing a software product; it is an adaptable methodology that has to be modified for the specific product. It is a modeling technique: the product is represented by UML diagrams. It is iterative and incremental and two-dimensional: it has five core workflows (technical context) and four phases (business context).*

### 3.2 Beş çekirdek iş akışı (core workflows)

### 3.2.1 Requirements workflow

**Amaç:** müşterinin *ihtiyaçlarını* (needs) belirlemek; *istediklerini* (wants) değil.

1. **Uygulama alanını (application domain) anla**: ürünün çalışacağı iş ortamını öğren.
2. **İş modelini (business model) kur**: müşterinin iş süreçlerini UML ile tanımla. Müşteri herhangi bir anda maliyeti haklı bulmazsa geliştirme hemen durur.
3. Müşterinin **kısıtlarını (constraints)** belirle: *deadline* (teslim tarihi; ürünler çoğu zaman *mission critical*), *parallel running* (eski sistemle paralel çalışma), *portability* (taşınabilirlik), *reliability* (güvenilirlik), *rapid response time* (hızlı yanıt), *cost* (müşteri bütçeyi nadiren söyler; teklif / ihale usulü kullanılır).

Bu kavram keşfinin (*concept exploration*) amacı müşterinin neye **ihtiyaç** duyduğunu bulmaktır.

### 3.2.2 Analysis workflow

**Amaç:** gereksinimleri **analiz etmek ve inceltmek (refine)**. Neden requirements'ta yapılmıyor? Çünkü requirements artifact'ları **müşterinin tamamen anlayacağı doğal dilde** olmak zorundadır ve bütün doğal diller belirsizdir. Örnek: "Veritabanından bir parça kaydı ve bir fabrika kaydı okunur. *Eğer* A harfinin hemen ardından Q harfini *içeriyorsa*, o parçanın o fabrikaya taşıma maliyetini hesapla." — "içeriyorsa"nın öznesi ne: parça kaydı mı, fabrika kaydı mı, veritabanı mı?

Bu yüzden iki ayrı iş akışı gerekir: requirements artifact'ları **müşterinin dilinde**, analysis artifact'ları ise tasarımcılar için **kesin ve yeterince tam** olmalıdır.

- **Specification document** bir **sözleşmedir (contract)**. "Optimal", "%98 tamamlanmış" gibi belirsiz ifadeler içermemelidir. **Çelişki (contradictions), eksiklik (omissions) ve tamamlanmamışlık (incompleteness)** olmamalıdır. Tam ve doğru spesifikasyon **test** ve **bakım** için şarttır.
- Müşteri spesifikasyonu imzaladıktan sonra ayrıntılı planlama ve kestirim başlar: **SPMP** (software project management plan) hazırlanır: **maliyet kestirimi (cost estimate), süre kestirimi (duration estimate), teslim edilecekler (deliverables), kilometre taşları (milestones), bütçe (budget)**. Bu, SPMP için **mümkün olan en erken zamandır**.

### 3.2.3 Design workflow

**Amaç:** analiz iş akışını, programcıların gerçekleştirebileceği forma gelene kadar inceltmek. Birçok **nonfunctional requirement** bu aşamada kesinleşir: programlama dili seçimi, yeniden kullanım, taşınabilirlik.

- Klasik: **architectural design** (ürünü modüllere ayırma) ve **detailed design** (her modülün veri yapıları ve algoritmaları).
- Nesne yönelimli: sınıflar **analysis** iş akışında çıkarılır, **design** iş akışında tasarlanır. Klasik mimari tasarım nesne yönelimli analizin, klasik ayrıntılı tasarım nesne yönelimli tasarımın bir parçasına karşılık gelir.
- **Tasarım kararları saklanmalıdır**: çıkmaz sokağa girildiğinde geri dönebilmek ve bakım ekibinin "tekerleği yeniden icat etmemesi" için.

### 3.2.4 Implementation workflow

**Amaç:** hedef ürünü seçilen programlama dilinde gerçekleştirmek. Büyük ürün alt sistemlere (subsystems), alt sistemler bileşenlere / kod artifact'larına (components, code artifacts) bölünür.

### 3.2.5 Test workflow

Test, her geliştiricinin, bakımcının ve **kalite güvence (SQA) grubunun** sorumluluğudur. **İzlenebilirlik (traceability)** başarılı testin önemli gereksinimidir: analysis artifact'larındaki her öğe requirements artifact'larındaki bir öğeye izlenebilmelidir (design ve implementation için de aynısı).

- **Requirements / analysis artifact'ları:** *review* (gözden geçirme) ile kontrol edilir; müşteri temsilcisi ve analiz ekibi bulunur. SPMP de benzer şekilde kontrol edilir (özellikle maliyet ve süre kestirimleri).
- **Design artifact'ları:** design review şarttır; müşteri temsilcisi genellikle bulunmaz.
- **Implementation artifact'ları:**
  - **Unit testing**: her bileşen gerçekleştirildiği anda test edilir.
  - **Integration testing**: her yinelemenin sonunda bileşenler birleştirilip test edilir.
  - **Product testing**: ürün tamamlanmış göründüğünde bütün olarak test edilir.
  - **Acceptance testing**: ürün müşterinin bilgisayarına kurulduktan sonra **müşteri** test eder.
  - COTS yazılımlarda: **alpha release** (geliştirme testleri sürerken olası müşterilere test için verilen sürüm) ve **beta release** (neredeyse tamamlanmış, ilk işletme kalitesindeki sürüm). Alfa/beta sitesi olmanın avantajı ürünü erken görmek ve rakiplerden önce davranmak; dezavantajı hatalı yazılımla uğraşmaktır.

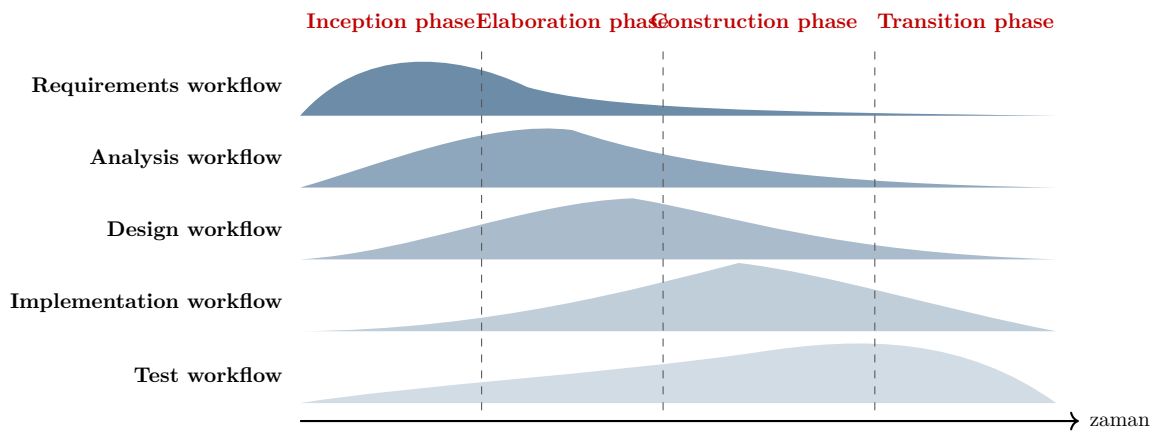
### 3.2.6 Postdelivery maintenance ve retirement

Teslim sonrası bakım geliştirmenin vazgeçilmez parçasıdır; **diğer bütün etkinliklerin toplamından fazla para** buna harcanır. Sorunların başlıca kaynağı her türlü **dokümantasyon eksikliğidir**. İki tür test gerekir: yapılan değişikliğin testi ve **regression testing** (bu yüzden önceki bütün test durumları ve beklenen sonuçları saklanmalıdır).

Yazılım şu durumlarda bakım yapılamaz hâle gelebilir: tasarımda köklü değişiklik; tamamen yeni donanım/işletim sistemi; eksik veya yanlış dokümantasyon; donanım değişikliği yüzünden sıfırdan yazmanın daha ucuz olması. Bunlar da aslında bakımdır (mevcut yazılımın yeniden yazılması). **Gerçek retirement** nadirdir: müşteri kurumun ürünün sağladığı işleve **artık hiç ihtiyacı kalmadığında** olur.

## 3.3 Unified Process'in dört fazı

Unified Process'te artırımlar **faz** olarak adlandırılır. Teoride herhangi sayıda artırım olabilir; pratikte geliştirme dört artırımdan oluşur:



Unified Process'teki her adım hem beş iş akışından birine hem de dört fazdan birine düşer. Neden iki kez düşünülür? **Workflow = adımın teknik bağlamı (technical context)**, **phase = adımın iş/ekonomik bağlamı (business context)**.

### 3.3.1 Inception phase (başlangıç fazı)

**Amaç:** önerilen yazılım ürününün **ekonomik olarak uygulanabilir (economically viable)** olup olmadığını belirlemek. Adımlar:

1. Alanı anla (understand the domain).
2. İş modelini kur (build the business model).
3. Projenin kapsamını daralt (delimit the scope): iş modelinin yazılımın kapsadığı alt kümesine odaklan.
4. İlk **business case**'i hazırlamaya başla. Cevaplanacak sorular: Ürün maliyet-etkin mi? Yatırımın geri dönüşü ne kadar sürer? Geliştirmezsek maliyeti ne olur? Pazarlanacaksa pazar araştırması yapıldı mı? Zamanında teslim edilebilir mi? Geç kalırsa etkisi ne olur?

**Riskler** ve nasıl azaltılacakları da belirlenir. **Üç ana risk kategorisi:**

1. **Technical risks** (ekip deneyimi, yeni donanım zamanında gelir mi, CASE araçları var mı),
2. **Gereksinimleri doğru belirleyememe riski** (requirements workflow doğru yapılarak azaltılır),
3. **Mimariyi doğru kuramama riski** (mimari yeterince sağlam olmayabilir).

Riskler **sıralanır**; kritik riskler önce azaltılır. Inception fazında az miktarda analiz ve tasarım yapılabilir; kodlama genellikle yapılmaz ama bazen **proof-of-concept prototype** yapılır. Test iş akışı fazın neredeyse başında başlar (gereksinimlerin doğruluğu). Başta tüm projeyi planlamaya yetecek bilgi olmadığından **yalnızca bir sonraki fazın (elaboration) planı** yapılır.

**Inception çıktıları (deliverables):** domain model'in ilk sürümü, business model'in ilk sürümü, requirements artifact'larının ilk sürümü, analysis artifact'larının ön sürümü, mimarinin ön sürümü, **ilk risk listesi**, use case'lerin ilk sıralaması, elaboration fazının planı, **business case'in ilk sürümü** (fazın asıl amacı).

### 3.3.2 Elaboration phase (ayrıntılandırma fazı)

**Amaç:** ilk gereksinimleri **inceltmek**; mimariyi inceltmek; riskleri izleyip önceliklerini güncellemek; business case'i inceltmek; **project management plan**'ı (SPMP) hazırlamak. Bu fazda requirements iş akışı neredeyse tamamlanır, analiz iş akışının neredeyse tamamı yapılır, mimari tasarıma başlanır.

**Elaboration çıktıları:** tamamlanmış domain model, business model, requirements ve analysis artifact'ları; mimarinin güncel sürümü; güncel risk listesi; **project management plan** (projenin geri kalanı için); tamamlanmış business case.

### 3.3.3 Construction phase (yapım fazı)

**Amaç:** ürünün **ilk işletme kalitesindeki (first operational-quality) sürümünü** üretmek; buna **beta release** denir. Vurgu **implementation ve test**tedir: modüllerin unit testi, alt sistemlerin integration testi, bütün sistemin product testi.

**Construction çıktıları:** ilk kullanıcı kılavuzu ve diğer kılavuzlar; bütün artifact'lar (beta sürümleri); tamamlanmış mimari; güncel risk listesi; güncel project management plan; gerekirse güncel

business case.

### 3.3.4 Transition phase (geçiş fazı)

**Amaç:** müşterinin gereksinimlerinin **gerçekten karşılandığından emin olmak**. Hatalar düzeltilir, bütün kılavuzlar tamamlanır, daha önce fark edilmemiş riskler bulunmaya çalışılır. Bu faz, beta sürümün kurulduğu sitelerden gelen **geri bildirimle** yürür.

**Transition çıktıları:** bütün artifact'ların son sürümleri, tamamlanmış kılavuzlar.

| Faz | Amaç | Ağırlıklı iş akışı |
|---|---|---|
| Inception | Ürün ekonomik olarak yapılabilir mi? İlk business case. | Requirements |
| Elaboration | Gereksinimleri ve mimariyi incelt, SPMP'yi çıkar. | Requirements + Analysis |
| Construction | İlk işletme kalitesinde sürüm (beta). | Implementation + Test |
| Transition | Müşteri gereksinimleri karşılandı mı? Hataları düzelt, kılavuzları tamamla. | Test (geri bildirim) |

**Çıkmış test sorusu**

"Which one is NOT a phase of Unified Process?" (Elaboration, Construction, **Testing**, Transition, Inception). Cevap: **Testing**. Test bir *iş akışıdır*, faz değildir. Fazlar ile iş akışlarını karıştırmayın.

## 3.4 Tek ve iki boyutlu yaşam döngüsü modelleri

- Geleneksel yaşam döngüsü (ör. waterfall) **tek boyutludur**: sadece fazlar ekseni vardır.
- Unified Process **iki boyutludur**: bir eksende iş akışları (teknik bağlam), diğerinde fazlar (iş bağlamı). Evolution-tree modeli de iki boyutludur.
- Neden bu karmaşıklık gerekli? İdeal dünyada her iş akışı bir sonrakinden önce biterdi. Gerçekte iş çok büyüktür; **Miller yasası** gereği iş artırımlara (fazlara) bölünmeli ve her artırımda iş bitene kadar yineleme yapılmalıdır. Başlangıçta gereksinimleri tamamlamaya yetecek bilgi yoktur. Ürün alt sistemlere, onlar da bileşenlere bölünür.
- Unified Process kaçınılmaz değişiklikleri (**moving target problem**, **kaçınılmaz hatalar**) iyi karşılar; onları *ortadan kaldırmaz*, *etkilerini azaltır*. Büyük bir problemi küçük, büyük ölçüde bağımsız alt problemler kümesi olarak ele almanın bugüne kadarki en iyi çözümüdür; gelecekte daha iyisiyle yer değiştirecektir.

**"Why do we need a two-dimensional model?"**

*Real projects are too large to be completed in one pass. Because of Miller's law the task is divided into increments (phases) and within each increment the workflows are iterated. A two-dimensional model shows both the workflows (technical context) and the phases (business context); it handles the moving target problem and the inevitable mistakes better, and faults are observed earlier.*

## 3.5 IEEE Software Project Management Plan (SPMP)

SPMP'nin özü (Chapter 3): **cost estimate, duration estimate, deliverables, milestones, budget**. IEEE Std 1058 bunun ayrıntılı bir çerçevesini verir (kitapta Section 9.5, Figure 9.9). Hoca **yalnızca 8 ana başlığı** sorar:

| # | Ana başlık | Alt başlıklar / içerik |
|---|---|---|
| 1 | **Overview** | 1.1 Project summary (purpose, scope and objectives; assumptions and constraints; project deliverables; schedule and budget summary), 1.2 Evolution of the project management plan |
| 2 | **Reference materials** | Uyulan dokümanlar ve standartlar |
| 3 | **Definitions and acronyms** | Tanımlar ve kısaltmalar (ör. MSG) |
| 4 | **Project organization** | External interfaces, internal structure, roles and responsibilities |
| 5 | **Managerial process plans** | Start-up plan (estimation, staffing, resource acquisition, staff training), work plan (work activities, schedule/resource/budget allocation), control plan (requirements, schedule, budget, quality control; reporting; metrics collection), risk management plan, project closeout plan |
| 6 | **Technical process plans** | Process model, methods/tools/techniques, infrastructure plan, product acceptance plan |
| 7 | **Supporting process plans** | Configuration management, testing, documentation, quality assurance, reviews and audits, problem resolution, subcontractor management, process improvement |
| 8 | **Additional plans** | Projeye özel ek planlar |

**Ezber ipucu**

"**O**verview – **R**eference – **D**efinitions – **O**rganization – **M**anagerial – **T**echnical – **S**upporting – **A**dditional": önce proje tanıtılır (1–3), sonra kim yapacak (4), sonra nasıl yönetilecek (5), teknik olarak nasıl yapılacak (6), destek süreçleri (7), ekler (8). IEEE çerçevesi her tür ürün için tasarlanmıştır, belirli bir yaşam döngüsü modeli dayatmaz ve Unified Process için idealdir (requirements control ve risk management bölümleri vardır).

**Eski sürüm: 5 başlıklı SPMP**

Klasördeki 2021 öğrenci notunda IEEE SPMP'nin **eski sürümü** (IEEE 1058.1-1987, kitabın eski baskıları) yazılı: **1 Introduction** (project overview, deliverables, evolution of the SPMP, reference materials, definitions and acronyms), **2 Project Organization** (process model, organizational structure, boundaries and interfaces, project responsibilities), **3 Managerial Process** (management objectives and priorities; assumptions, dependencies and constraints; risk management; monitoring and controlling mechanisms; staffing plan), **4 Technical Process** (methods, tools and techniques; software documentation; project support functions), **5 Work Packages, Schedule and Budget**. Hoca videoda kitabın 8. baskısındaki **8 başlıklı** çerçeveyi gösterdi; sınavda 8 başlığı yazın.

## 3.6 Yazılım sürecini iyileştirme (Improving the software process)

ABD Savunma Bakanlığı girişimiyle kurulan **Software Engineering Institute (SEI)**'ye göre yazılımdaki temel sorun **sürecin kötü yönetilmesidir**. Hocanın vurgusu: *süreç kaliteliyse ürün de kaliteli olur*; bu standartlar ürünle değil **üretim süreciyle** ilgilenir, her şeyin belgelenmesini ister: *belge yoksa ilerlemeyi izleyemezsin; izleyemezsen başarını ölçemezsin; ölçemezsen kendini geliştiremezsin.*

- **CMM (Capability Maturity Model)** beş olgunluk seviyesi: **1 Initial** (ad hoc, kriz yönetimi), **2 Repeatable** (temel proje yönetimi; önceki benzer projeler tekrarlanabilir), **3 Defined** (süreç tamamen belgelenmiş), **4 Managed** (nitel ve nicel hedefler, ölçüm), **5 Optimizing** (sürekli süreç iyileştirme).
- **ISO 9000 serisi**: kalite sistemleri standardı (yazılım için ISO 9000-3).
- **ISO/IEC 15504 (SPICE)**: süreç değerlendirme standardı.

- Hoca: *"you can read it but the ISO 25000 series are more important for me"* ⇒ ayrıntı için Konu 5 (ISO/IEC 25010).

## 3.7 Kısa tanım soruları

**Workflow vs phase** A workflow is the technical context of a step; a phase is the business context of a step. Every step falls into one workflow and one phase.

**Business case** A document describing the scope of the product and its financial details (revenue projections, cost estimates or cost–benefit analysis); its initial version is the overall aim of the inception phase.

**Beta release** The first operational-quality version produced at the end of the construction phase.

**Traceability** Every item in an artifact must be traceable to an item in the previous artifact.

**Specification document** A contract between client and developer; it must not contain contradictions, omissions or incompleteness.

**Three major risks** Technical risks, the risk of not getting the requirements right, and the risk of not getting the architecture right.